

Transformer Fundamentals

A smaller set of notes than the distributed-training curriculum this accompanies — four focused scripts (~650 lines), each verified by actually running it: a GPT written from scratch, the training/inference mode split that motivates KV caching, a real measured cache speedup on GPT-2, and a basic interpretability technique.

M4 MacBook · PyTorch · HuggingFace Transformers · TransformerLens

§1

1. Causal self-attention, from scratch

~150 lines: multi-head attention, pre-norm residual blocks, weight tying.

A decoder-only GPT (`MiniGPT`) implementing the standard architecture with no HuggingFace import — causal masking inside attention, pre-norm residual blocks (`x + f(norm(x))`, not `norm(x + f(x))` — the ordering that keeps gradients well-behaved as depth grows), and weight tying between the input embedding and output projection.

```
class CausalSelfAttention(nn.Module):
    def forward(self, x, *, past_kv=None, use_cache=False):
        b, t, c = x.shape
        qkv = self.qkv(x).reshape(b, t, 3, self.n_heads, self.d_head)
        q, k, v = qkv.unbind(dim=2)
        q, k, v = q.transpose(1, 2), k.transpose(1, 2), v.transpose(1, 2)

        if past_kv is not None:
            pk, pv = past_kv
            k = torch.cat([pk, k], dim=2) # append, don't recompute — the KV cache
            v = torch.cat([pv, v], dim=2)

        attn = (q @ k.transpose(-2, -1)) / math.sqrt(self.d_head)
        if past_kv is None:
            mask = torch.triu(torch.ones(t, t, dtype=torch.bool), diagonal=1)
            attn = attn.masked_fill(mask, float("-inf"))
        # else: nothing to mask — a single new query can see the full cache
```

minimal_gpt.py

Note the `past_kv` parameter and the branch that skips masking when it's present — this model implements its own KV cache from the start (§3), not bolted on after. Verified: trained 200 steps on a tiny char-level corpus, loss 79.3 → 0.78, and generated a legible (if imperfect) continuation from the prompt `"The cat"`.

§2

2. Training vs. inference: parallel vs. serial

The distinction that makes a KV cache necessary in the first place.

Training processes an entire sequence in one forward pass. Teacher forcing means the target at every position is already known, so the causal mask alone is enough to prevent position i from seeing positions $>i$ — no sequential dependency, fully parallelizable across the sequence dimension. **Inference** can't do this: token $i+1$ depends on the model's own prediction at step i , which doesn't exist until it's generated. Autoregressive generation is inherently serial, one token at a time.

```
=== TRAINING MODE (one parallel forward) ===
Input tokens:  torch.Size([1, 7])
Logits:        (1, 7, 34) ← all positions at once
Next-token loss (teacher forcing): 1.561

=== INFERENCE MODE (serial, one token per step) ===
Step 0 prefix: 'The cat'
Step 1: appended ' ' → 'The cat '
Step 2: appended 'm' → 'The cat m'
Step 3: appended 'a' → 'The cat ma'
...
```

`infer_demo.py` — actual run, same checkpoint, both modes side by side

This split is the entire reason KV caching exists: in serial generation, step $i+1$'s attention still needs the keys/values for every earlier token. Recomputing them from scratch each step — the naive approach — is pure waste, since they haven't changed.

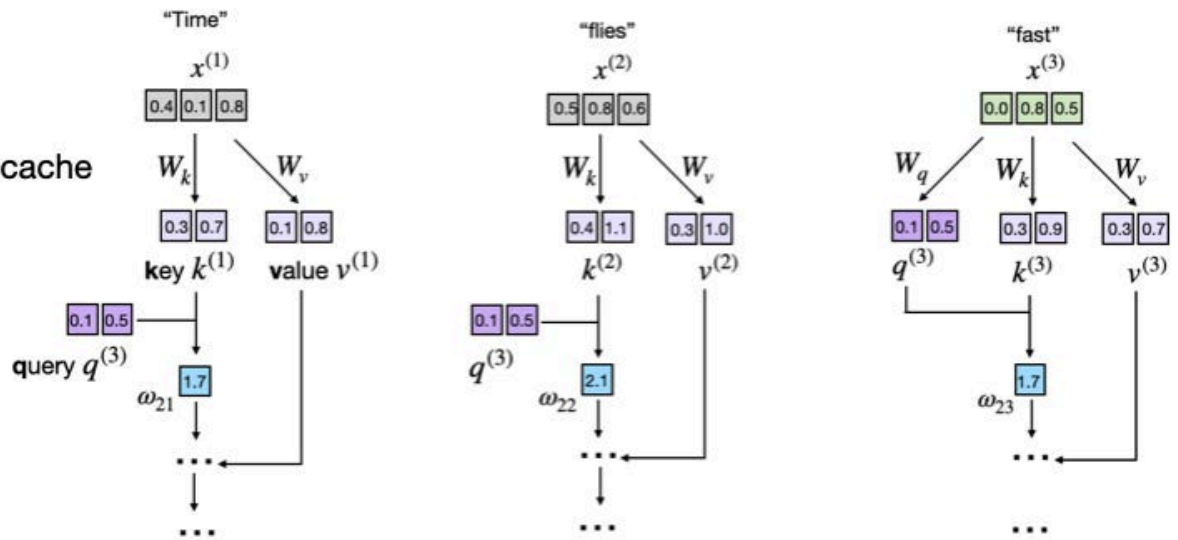
§3

3. KV cache

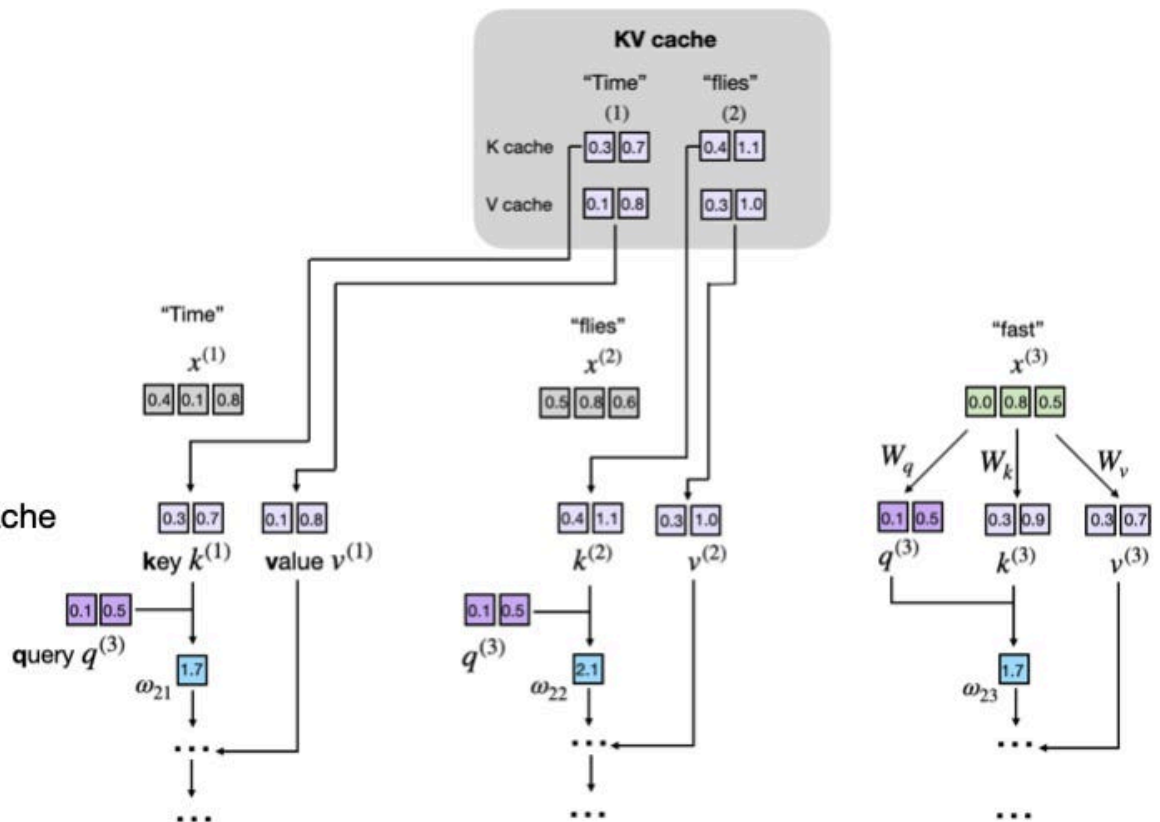
$O(n^2)$ recomputation vs. $O(n)$ incremental — measured, not asserted.

Without a cache, generating token n means recomputing keys and values for all $n-1$ preceding tokens from scratch, every step — the per-step cost grows with sequence length, and the total cost across a full generation is **$O(n^2)$** . Caching stores each token's key/value the moment it's computed and reuses it forever after; each new step only computes the *new* token's K/V and appends — **$O(n)$** incremental cost per step.

Without KV cache



With KV cache



Without caching, generating token 3 recomputes $k^{(1)}$, $v^{(1)}$ and $k^{(2)}$, $v^{(2)}$ from scratch — work already done twice before. With caching, those are stored once and reused; only the new token's own K/V get computed. [Sebastian Raschka, "Coding the KV Cache in LLMs"](#)

```
def generate_with_cache(model, input_ids, new_tokens):
    ids, past = input_ids.clone(), None
    for step in range(new_tokens):
        ctx = ids if past is None else ids[:, -1:] # only the new token after step 0
```

```

    out = model(ctx, past_key_values=past, use_cache=True)
    past = out.past_key_values
    next_id = out.logits[:, -1].argmax(dim=-1, keepdim=True)
    ids = torch.cat([ids, next_id], dim=1)
return ids

```

kv_cache_demo.py

GENERATION	24 NEW TOKENS, REAL GPT-2
without cache	5.338s
with cache	0.526s
speedup	10.15×

Real HuggingFace `gpt2`, measured directly, not looked up — the gap widens further as sequence length grows, since the no-cache cost is quadratic and the cached cost is linear.

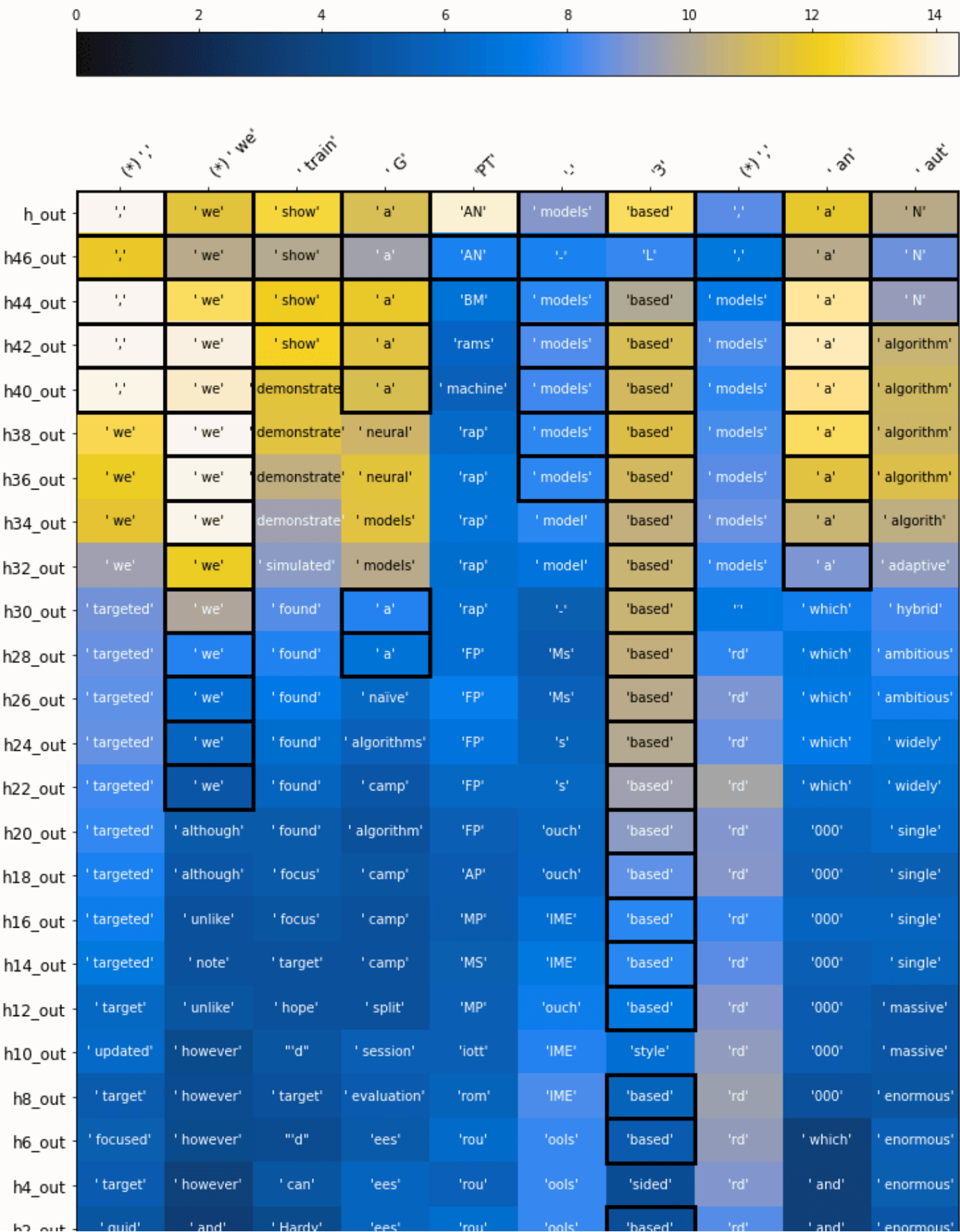
§4

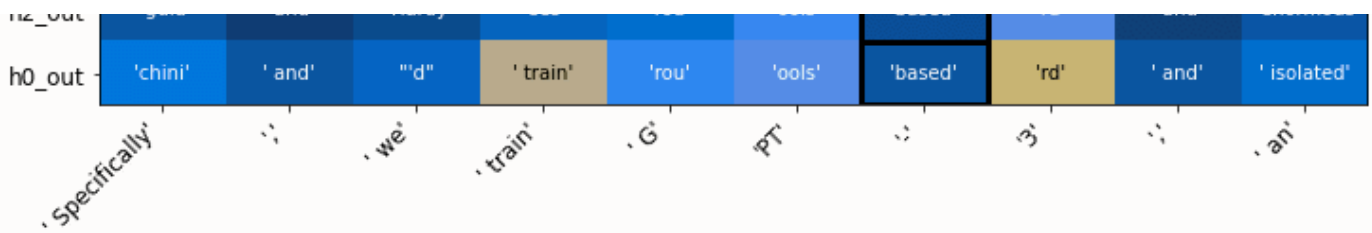
4. Logit lens

Reading a prediction out of a layer the model was never trained to predict from.

The residual stream is what every layer reads from and writes to; the final layer norm plus unembedding matrix (W_U) is what turns the *last* layer's residual stream into next-token logits. The logit lens (nostalgebraist, 2020) applies that same unembedding to the residual stream at every intermediate layer, not just the last — treating each layer as if it were the final one. There's no guarantee this produces anything coherent (nothing trained the model to make intermediate layers directly legible this way), but empirically, predictions sharpen from diffuse to confident across depth.

model's top token and its logit





The original figure: each column is a token position, each row a layer (bottom = earliest); each cell shows the top-predicted token if generation stopped at that layer. Predictions visibly stabilize with depth. [nostalgebraist, "interpreting GPT: the logit lens," 2020](#)

```
_, cache = model.run_with_cache(tokens)
for layer in range(model.cfg.n_layers):
    resid = cache[f"blocks.{layer}.hook_resid_post"][0, final_pos]
    logits = model.unembed(resid) # apply the FINAL unembedding early
    probs = torch.softmax(logits, dim=-1)
```

logit_lens_demo.py – via TransformerLens's activation cache

Run on GPT-2-small, prompt "The cat sat on the":

```
Layer 0 → top token: ' same' (p=0.981)
Layer 5 → top token: ' same' (p=0.986)
Layer 6 → top token: ' shoulders' (p=0.739)
Layer 8 → top token: ' sidelines' (p=0.774)
Layer 10 → top token: ' sofa' (p=0.999)
Layer 11 → top token: ' sofa' (p=0.880)
```

Early layers default to a generic high-frequency completion; the model only commits to something contextually grounded ("sofa" — a plausible thing to sit *on*) in the second half of the network.

WHAT RUNNING IT CAUGHT

`model.to_str_tokens(tokens)[0]` was indexing into the token list, not the batch dimension — since batch size is 1, this silently printed only the first token (always `<|endoftext|>`, TransformerLens's auto-prepended BOS) instead of the actual prompt tokens. The real computation below it was unaffected — this only broke one diagnostic log line — but it's exactly the kind of thing that looks fine until you actually read the output. Fixed: drop the `[0]`.

Full code: github.com/longyi1207/ai_lab/transformer_lab · distributed-training companion: [dist_training_lab](#)

